SE 2228-Analysis and Design of Algorithms

Analysis-1 : Basic Concepts,Big O

Review of Basic Concepts

- Suppose we have two algorithms, how can we tell which one is better?
- We could implement both algorithms, run them both
  - Experimental evaluation : Expensive and error prone
- Preferably, we should analyze them mathematically
  - *Algorithm analysis*
- For this ,we need to use description of the algorithm together with the properties of an associated data structure in a machine-independent way

Running Time

How to quantify the machine independent running time of an algorithm?

- Count the Number of basic steps that are executed
  - Most statements roughly require the same amount of time
  - We can be more exact if need be
- Obtain Worst case and average case numbers of steps
- Using standard time units (sec, msec…) is not appropriate-machine dependency!!!
- Counting all operations is not appropriate
- **Reasonable Approach: Identify and count the basic operation(s) in the algorithm**

Asymptotic Performance

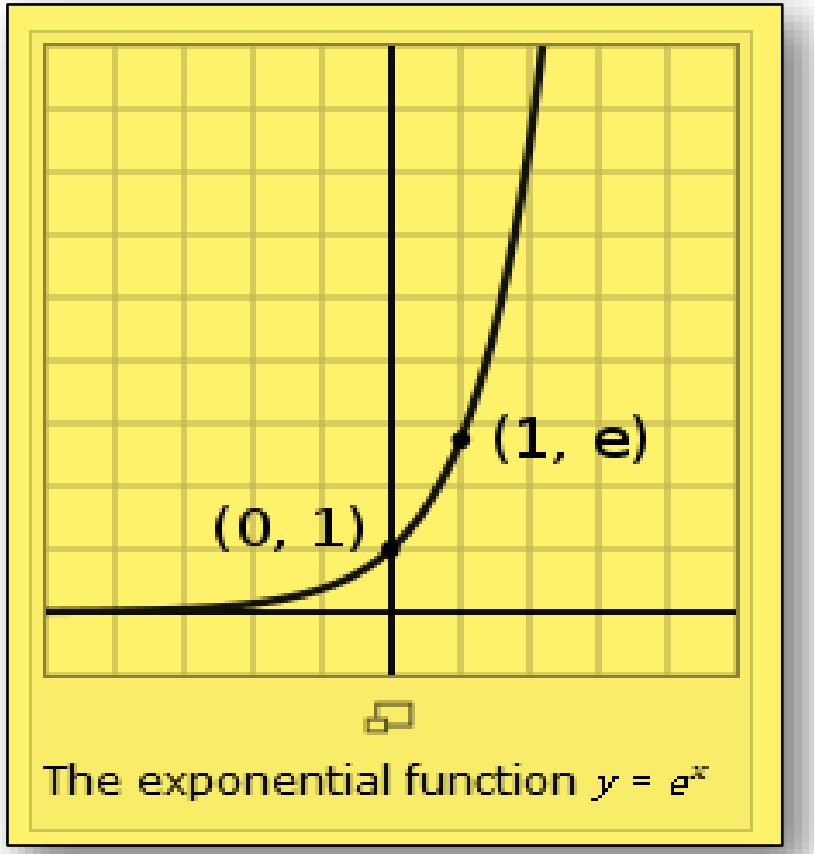
- *Asymptotic performance*: How does an algorithm behave as the problem size gets very large?
    - Time efficiency
    - Storage requirements
    - Network usage
  - Remember that we use the RAM model:
    - All memory cells are equally expensive to access
    - No concurrent operations
    - All reasonable instructions take unit time
      - Except, of course, function calls
  - Usually, time efficiency is more important.

Measuring Efficiency :Types of Mathematical Functions

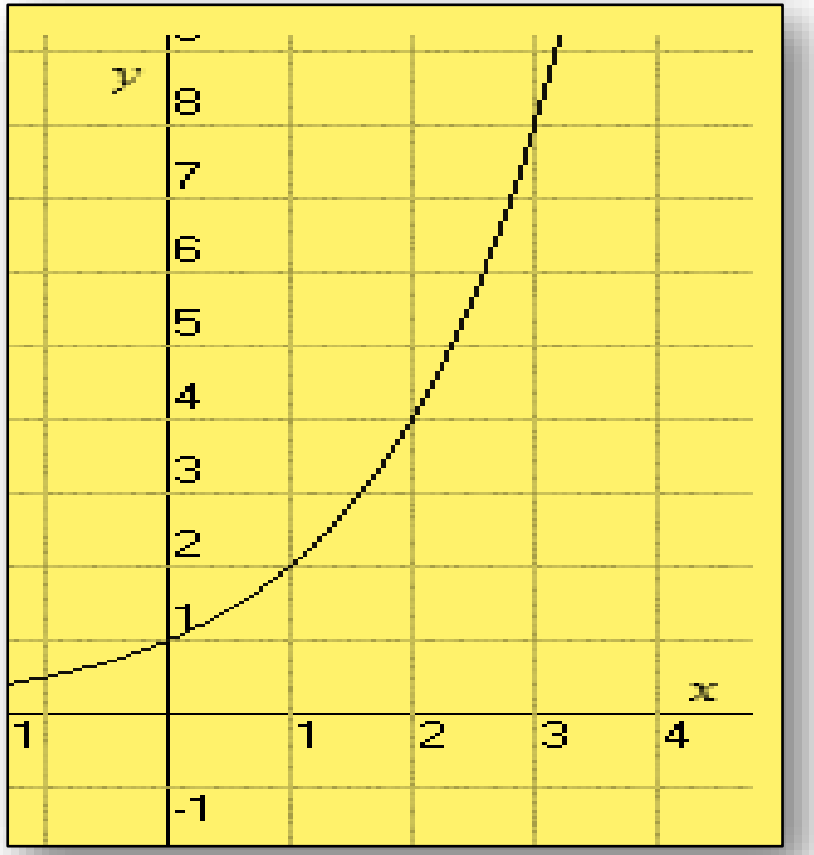
$y = x^a$: power function

$y = a^x$: exponential function

The Exponential Function: $y = b^x$, $b>1$, b is base

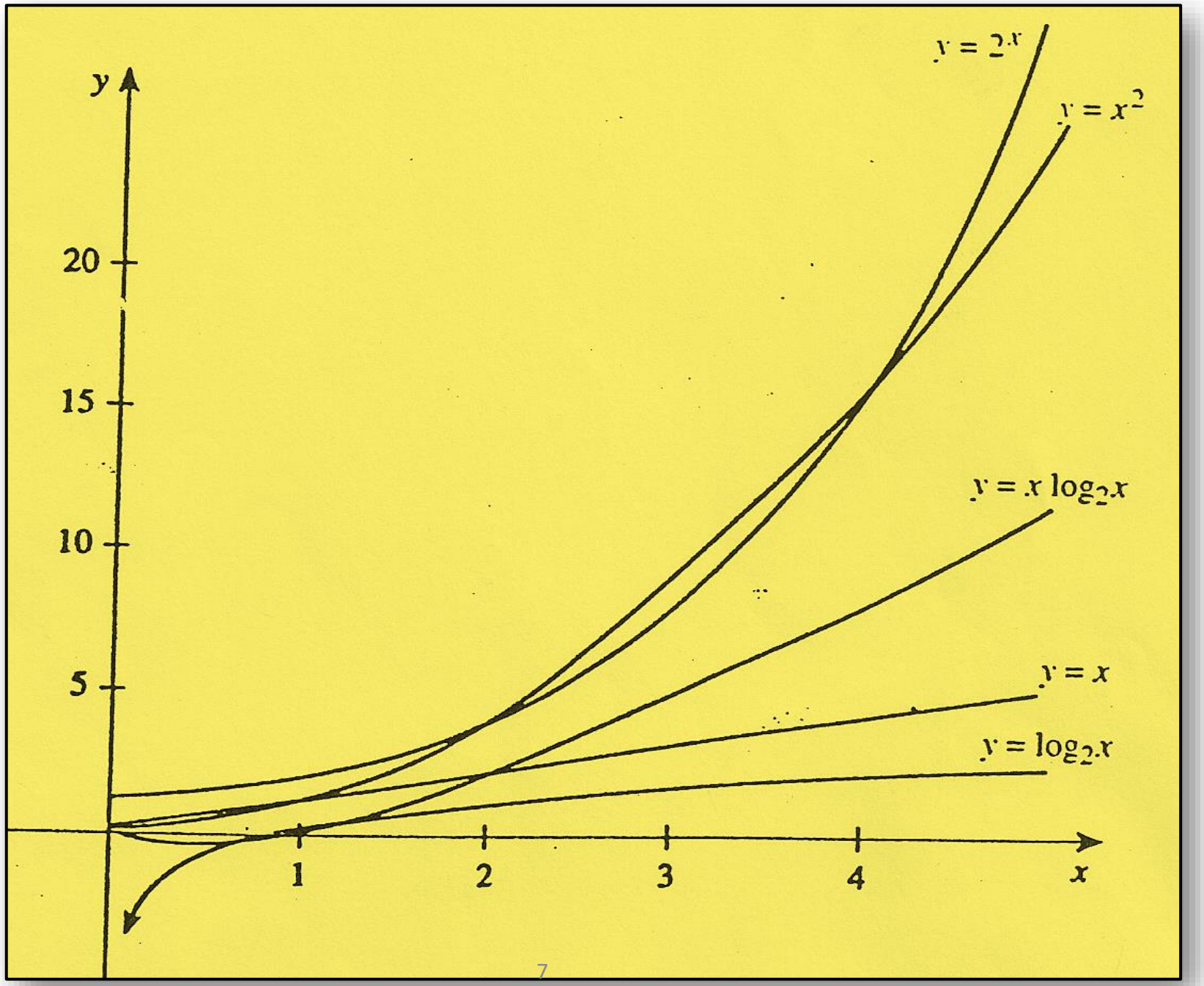


$y=e^x$



$y=2^x$

Comparison of Functions



Comparison of Functions for Increasing n

logarithmic power exponential

	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

The last line for n= 100 : The value of 2^n is ~ 10^{30}

Assume a function represents the number of operations executed and one operation takes 1 ns. Time needed for input size n:

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3 nsec	0.01 μ	0.02 μ	0.06 μ	0.51 μ	0.26 μ
16	4 nsec	0.02 μ	0.06 μ	0.26 μ	4.10 μ	65.5 μ
32	5 nsec	0.03 μ	0.16 μ	1.02 μ	32.7 μ	4.29 sec
64	6 nsec	0.06 μ	0.38 μ	4.10 μ	262 μ	5.85 cent
128	0.01 μ	0.13 μ	0.90 μ	16.38 μ	0.01 sec	10^{20} cent
256	0.01 μ	0.26 μ	2.05 μ	65.54 μ	0.02 sec	10^{58} cent
512	0.01 μ	0.51 μ	4.61 μ	262.14 μ	0.13 sec	10^{135} cent
2048	0.01 μ	2.05 μ	22.53 μ	0.01 sec	1.07 sec	10^{598} cent
4096	0.01 μ	4.10 μ	49.15 μ	0.02 sec	8.40 sec	10^{1214} cent
8192	0.01 μ	8.19 μ	106.50 μ	0.07 sec	1.15 min	10^{2447} cent
16384	0.01 μ	16.38 μ	229.38 μ	0.27 sec	1.22 hrs	10^{4913} cent
32768	0.02 μ	32.77 μ	491.52 μ	1.07 sec	9.77 hrs	10^{9845} cent
65536	0.02 μ	65.54 μ	1048.6 μ	0.07 min	3.3 days	10^{19709} cent
131072	0.02 μ	131.07 μ	2228.2 μ	0.29 min	26 days	10^{39438} cent
262144	0.02 μ	262.14 μ	4718.6 μ	1.15 min	7 mnths	10^{78894} cent
524288	0.02 μ	524.29 μ	9961.5 μ	4.58 min	4.6 years	10^{157808} cent
1048576	0.02 μ	1048.60 μ	20972 μ	18.3 min	37 years	10^{315634} cent

Table 1.1 Running times for different sizes of input. "nsec" stands for nanoseconds, "μ" is one microsecond and "cent" stands for centuries.

Age of the Universe

- From the table next, the last line is for n= 100

  The value of 2^n for n=100 is ~ 10^{30}

  How big is this value ?
- The estimated age of the Universe from the "Big Bang":

~ 13.7 *10^9 =13.7 billion years.

→4.32×10^{17} seconds

~ 10^{23} Microseconds = 10^{26} nanoseconds.

→A nanosec. Speed computer could not have solved this kind of problem during the entire lifetime of the universe!

	lg n	n	n lg n	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

How to Define Time Efficiency

- Problematic to measure the *absolute* running time
  - Depends on the size of the instance
  - For instances of the same size, may depend on:
    - Computer
    - Programming language, compiler,operating system
    - Details of implementation, skills of programmer
    - ……..
- Therefore, the absolute time measures are not appropriate and not used.

How to Define Time Efficiency

- We want the measure to be machine-independent, language-independent, etc. , and should survive technological advances.
- We are mainly concerned about large values of n.
- We do not use the absolute time, but rather use *rate/order of growth*.
  - Recall that the efficiency functions are non-decreasing
- Order is an asymptotic notation – we are only interested what happens for large n.

How to Define Time Efficiency

- Assume we have come up with some constant $c > 0$, such that the running time is at most cn^2. The value of c becomes inconsequential as n gets bigger and bigger.
- If we compare algorithms A and B that take times cn^2 and dn^3:

$t_B/t_A = dn^3/cn^2 = (d/c) * n$;

For big n, the ratio d/c also has insignificant effect.

- Therefore, we say A is of order n^2, and B is of order n^3, disregarding the constants.
- If two algorithms are of the same order, only then we may consider the constants to determine which algorithm is preferable.

Using the Input Size

- Time efficiency → the number of repetitions of the basic operations as a function of input size
- Input size is influenced by
  - the data representation, e.g. matrix, list , tree…
  - the basic operations of the algorithm, e.g. computations, string operations,pointer usage…
  - the properties of the objects in the problem, e.g. checking if a given integer is a prime number

Basic Operations :

- Applied to all input items in order to carry out the algorithm
- Contribute most towards the running time of the algorithm

Examples

Problem	Size of input
Find x in an array	The number of the elements in the array
Multiply two matrices	The dimensions of the matrices
Sort an array	The number of elements in the array
Traverse a binary tree	The number of nodes
Solve a system of linear equations	The number of equations, or the number of the unknowns, or both

Time Cost of an Algorithm

- Exact formula : $C(n) = n(n-1)/2$
- Approximate formula: indicating order of growth with unknown multiplicative constant : $C(n) \approx c\, n^2$

Let

$T(n)$: running time

c_{op} : execution time cost for a basic operation

$C(n)$: number of times the basic operation is executed

Then for a basic operation we have:

$$T(n) \approx c_{op}\, C(n)$$

Growth Rates

In general, we only worry about growth rates because:

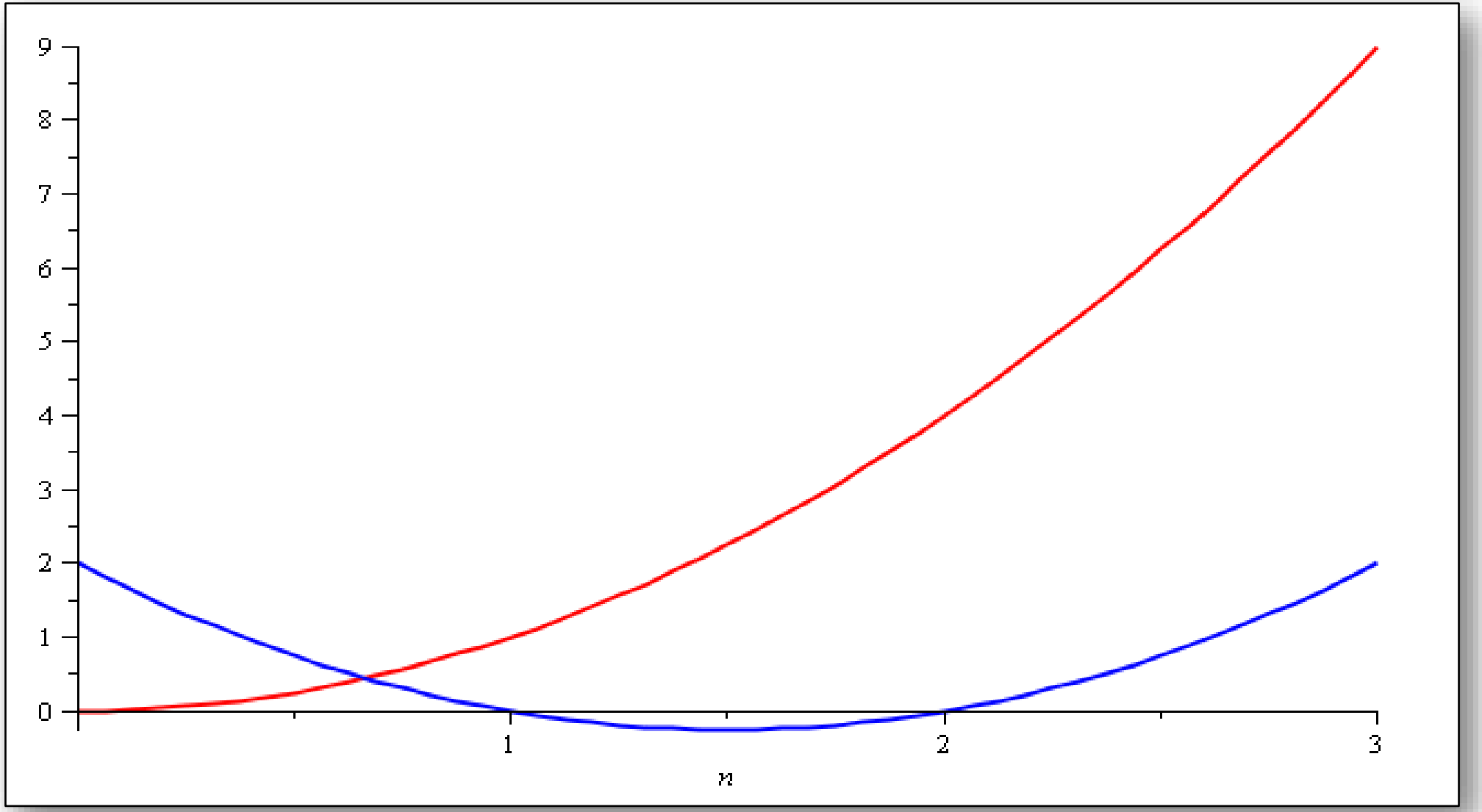
1. Our main objective is to analyze the cost performance of algorithms.
2. Another obstacle to having the exact cost of algorithms is that sometimes the algorithms are quite complicated to analyze.
3. When analyzing an algorithm, we are not that interested in the exact time the algorithm takes to run .

→What makes one algorithm more desirable than another is its growth rate relative to the other algorithm's growth rate.

Similar Growth Rates :Quadratic Growth

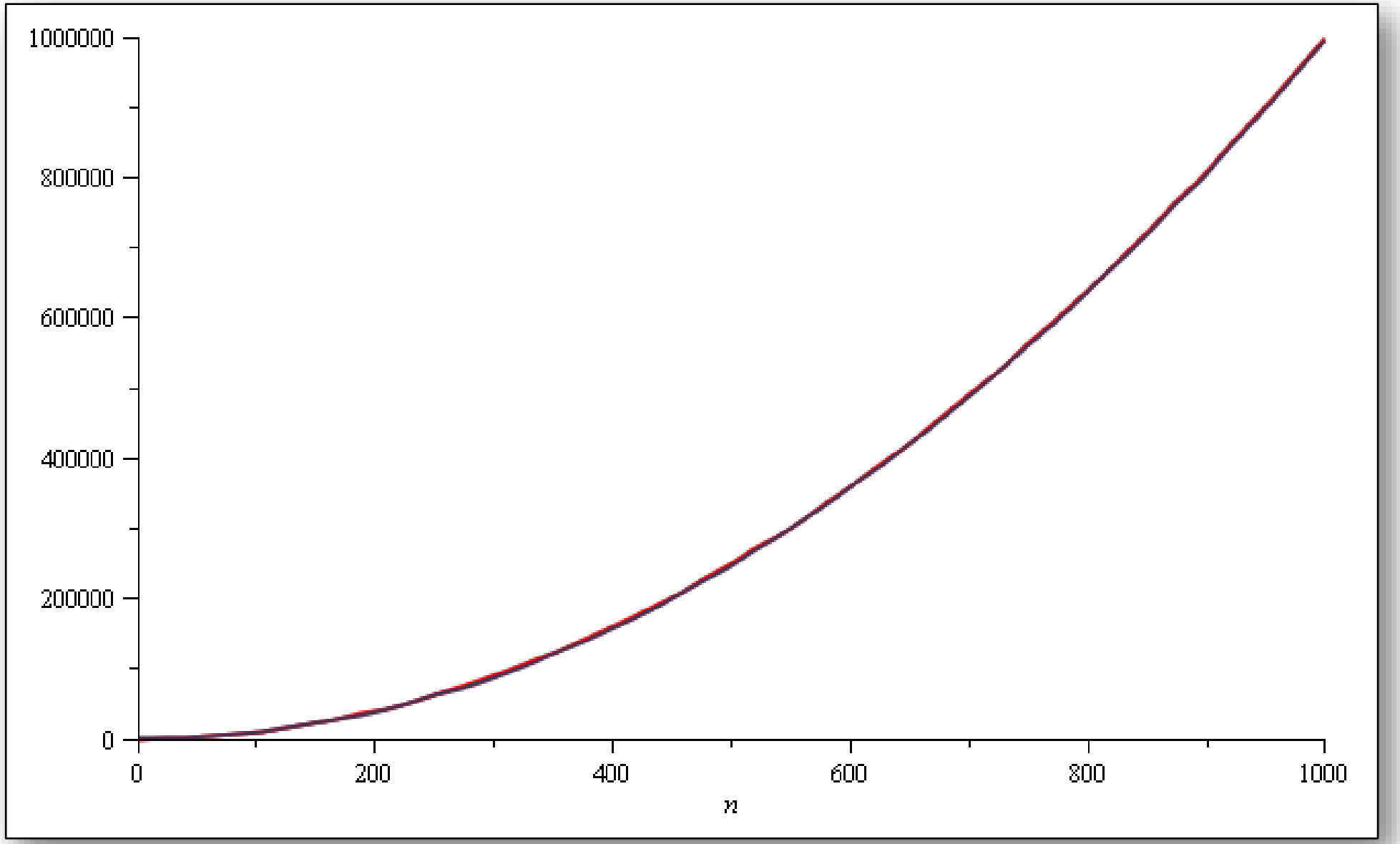
The meaning of asymptotic growth: Consider the two functions: $f(n) = n^2$ and $g(n) = n^2 - 3n + 2$

Around n = 0,1,2,3… they look very different

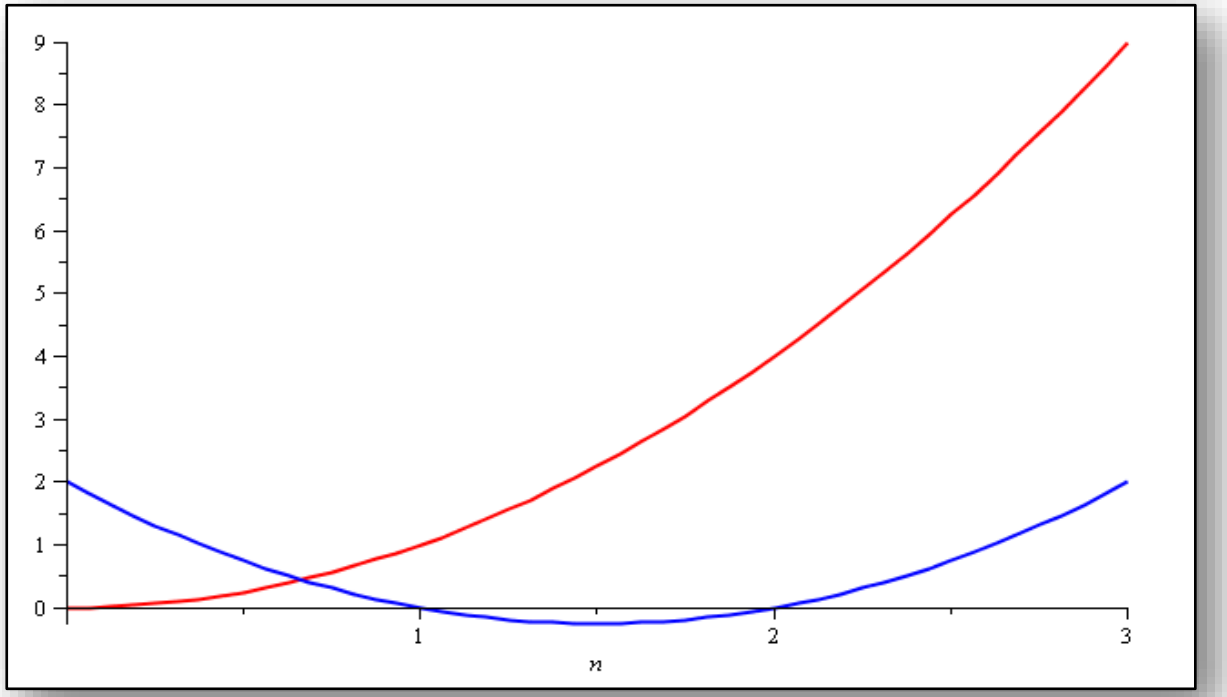


Similar Growth Rates :Quadratic Growth

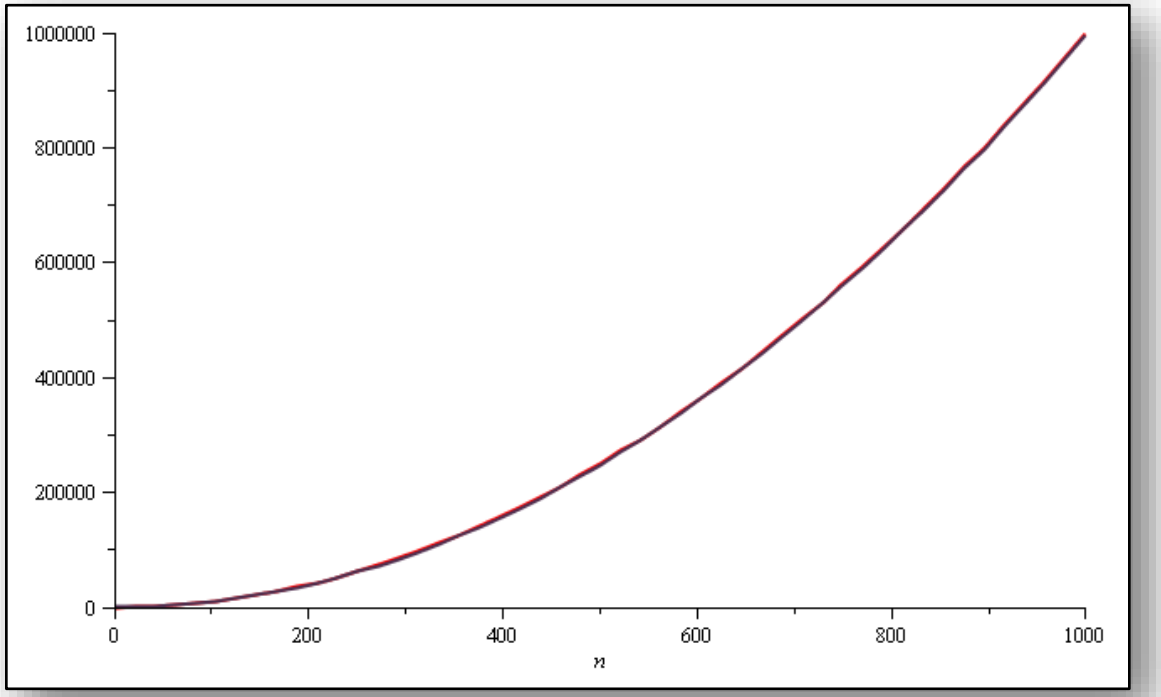
Yet on the range $n = [0, 1000]$, they are (relatively) indistinguishable:



Similar Growth Rates :Quadratic Growth



n is small



n gets larger and larger

Similar Growth Rates :Quadratic Growth

The absolute difference is large, for example,

$$f(1000) = 1\ 000\ 000$$

$$g(1000) = \ \ 997\ 002$$

But the relative difference is very small:

$$\left|\frac{f(1000) - g(1000)}{f(1000)}\right| = 0.002998 < 0.3\%$$

This difference goes to zero as $n \to \infty$

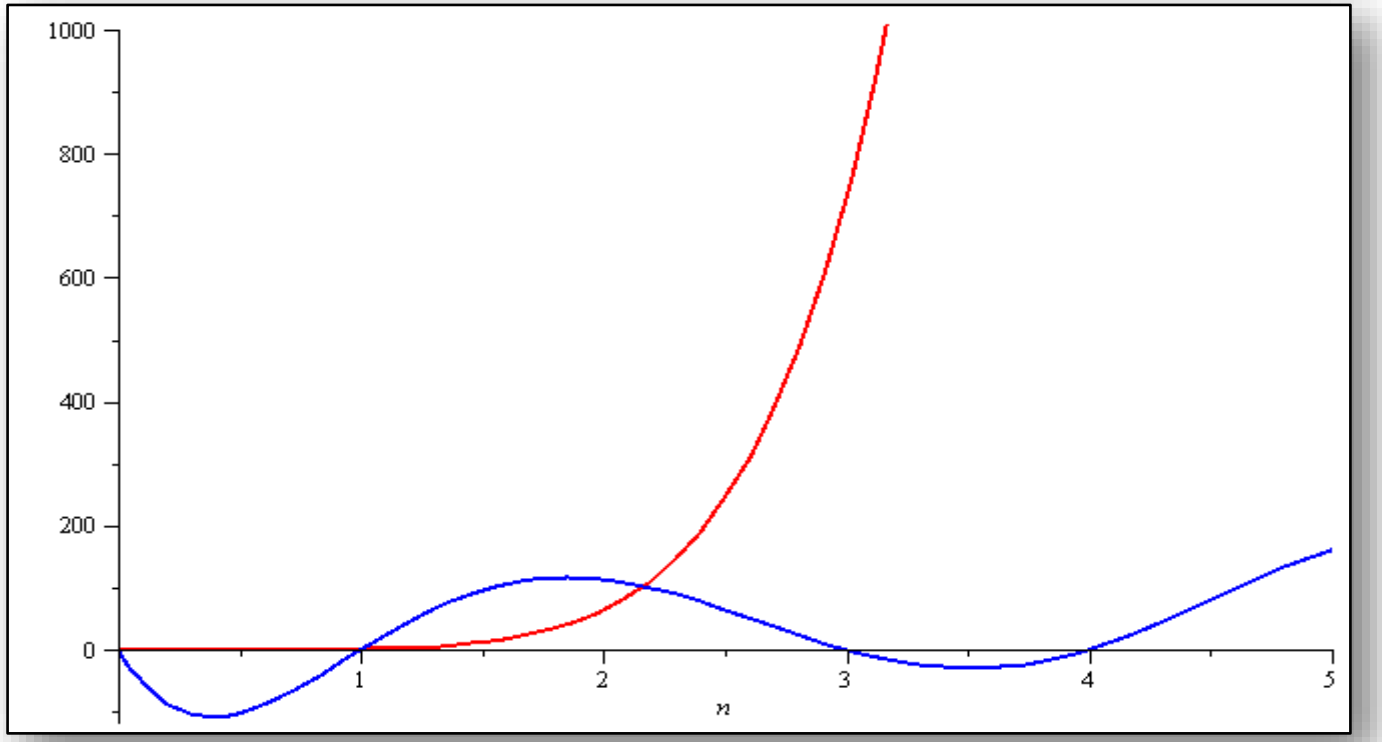
→ So, Asmptotically, the behavior of f(n) and g(n) is the same.

Polynomial Growth

To demonstrate with another example,

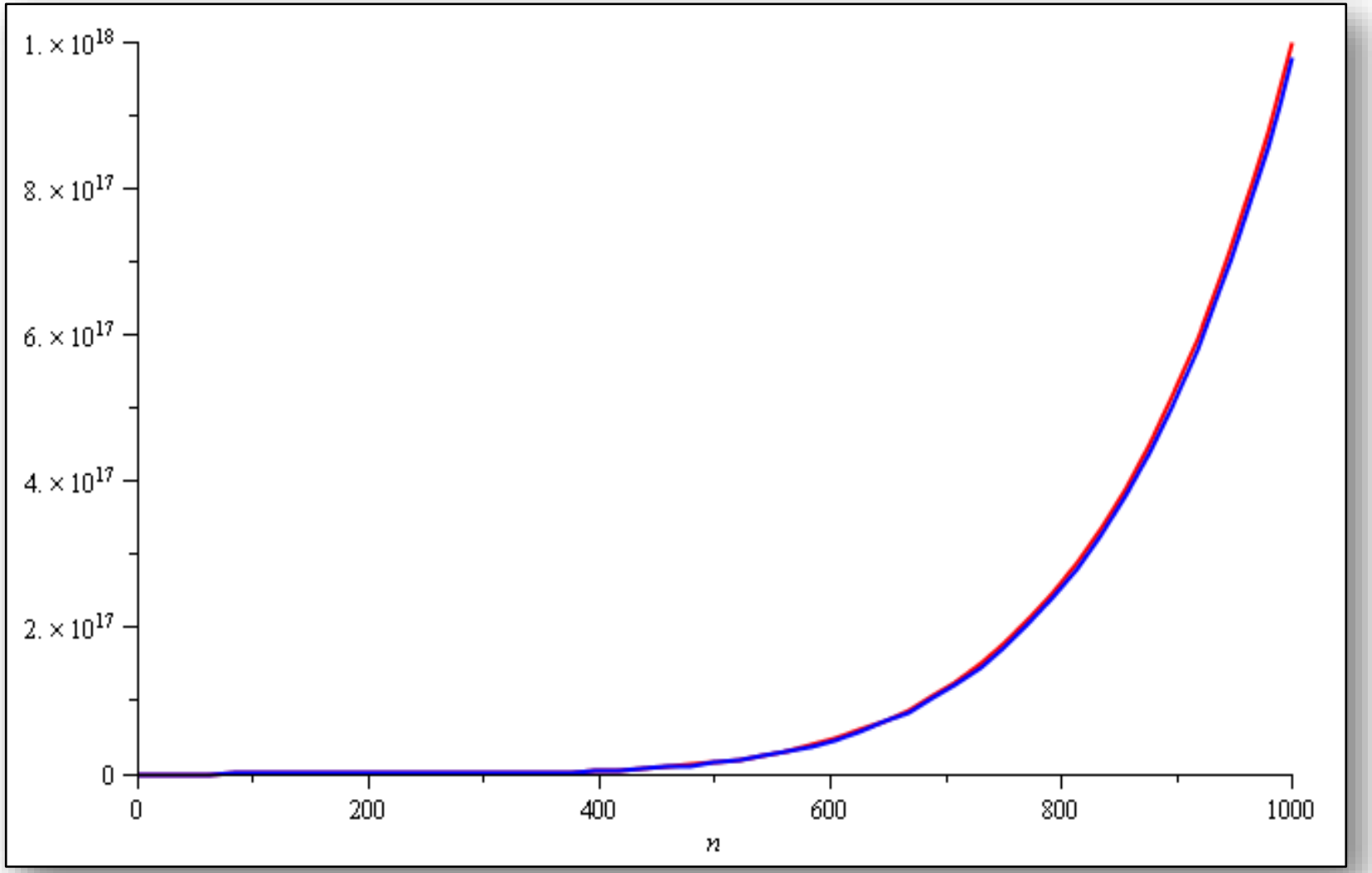
$f(n) = n^6$ and $g(n) = n^6 - 23n^5 + 193n^4 - 729n^3 + 1206n^2 - 648n$

Around $n = 0,1,2,3\ldots$ they are very different



Polynomial Growth

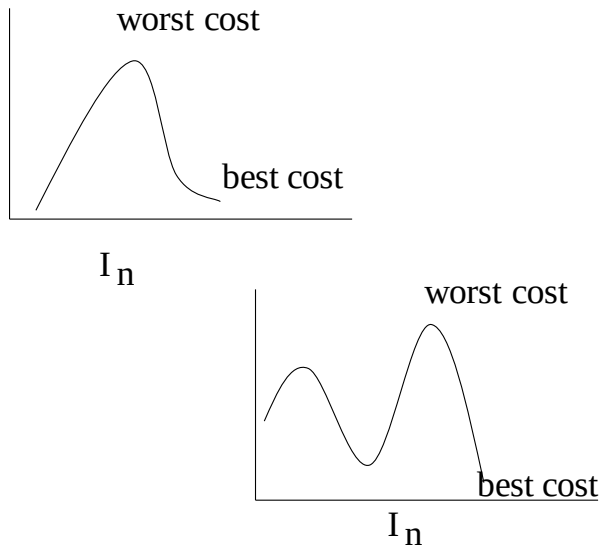
Still, around $n = 1000$, the relative difference is less than 3%



Reminder : Different Notions of Complexity

Worst case complexity of an algorithm A :
the maximum number of computational steps required for its execution over all the inputs of the same size. It provides an upper bound for an algorithm. The worst that can happen given the most difficult instance: the pessimistic view.

Best case complexity of an algorithm A -**the minimum number of computational steps required for its execution over all the inputs of the same size.** The most optimistic view of an algorithm– it tells us the least work a particular algorithm could possibly get away with the easiest input of a given size.



I_n – all possible instances of size n

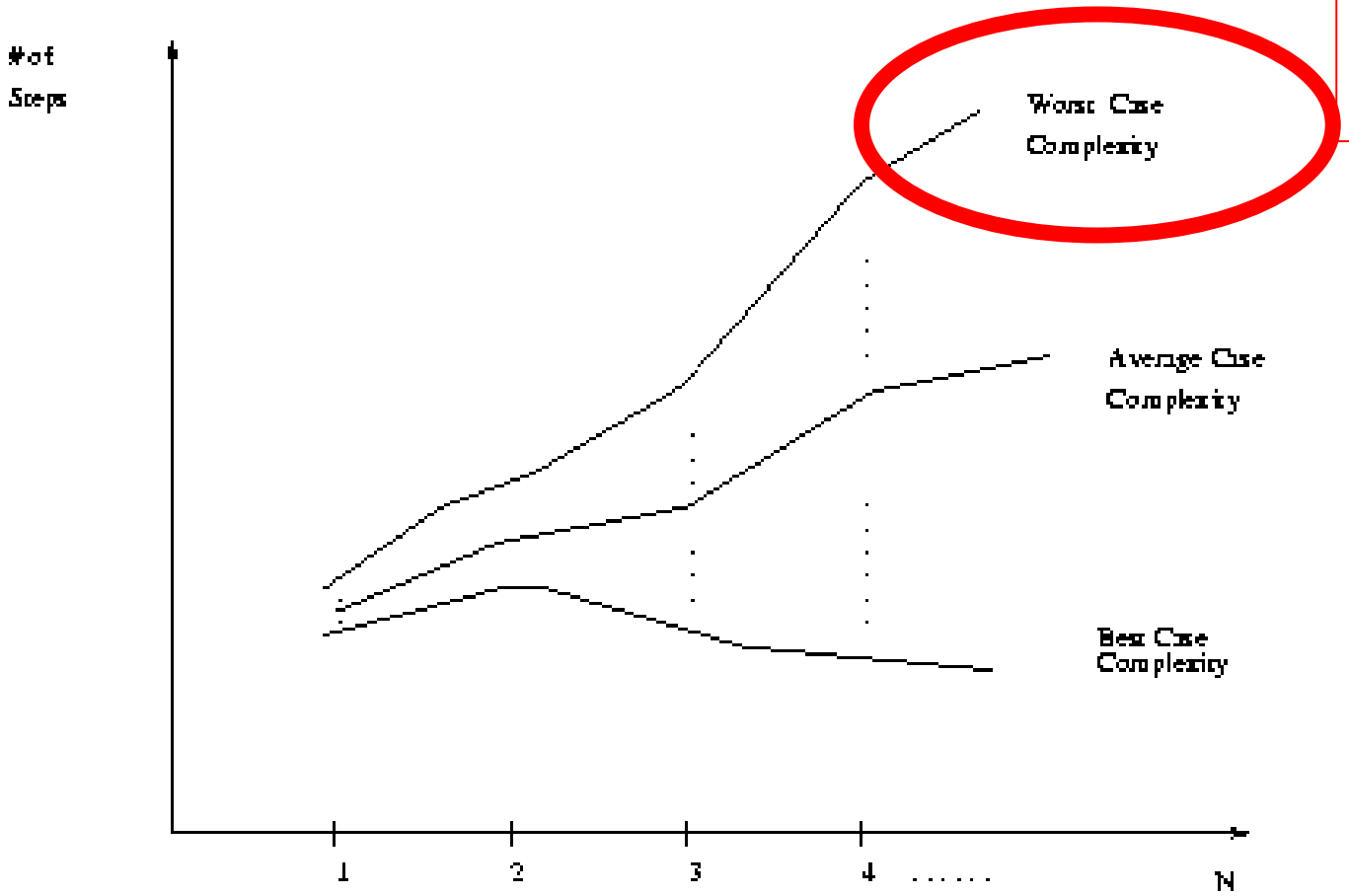
Average Case Complexity

Average case complexity of an algorithm A - i.e., the average amount of resources the algorithm consumes assuming some plausible frequency of occurrence of each input.

- Determining the average cost is much more difficult than determining either the worst or best-cost.

→ we have to assume a given probability distribution for the types of inputs we may get.

Different Notions of Complexity



We perform upper bound analysis on algorithms.

In general, this is the notion that we use to characterize the complexity of algorithms

Example : Sequential (Linear) Search

- ***Problem:*** Given a list of *n* elements and a search key *K*, find an element equal to *K*, if any.
- ***Informal Algorithm:*** Scan the list and compare its successive elements with *K* until either a matching element is found (*successful search*) or the list is exhausted

  (*unsuccessful search*)

Searching : Sequential Search

Start from the leftmost element of arr[], one by one compare x with each element of arr[]

```
// Linearly search x in arr[].  If x is present then return its
// location,  otherwise return -1
int search(int arr[], int n, int x)
{
   int i;
         for (i=0; i<n; i++)
            if (arr[i] == x) return(i);
         return(-1);
}
```

Complexities : ?

Sequential Search

Number of comparisons:

Worst case: $c_n = n$

$c_n = O(n)$

x is either not in the list or it is in the last position

Best case: $c_n = 1$

$c_n = O(1)$

x is the first element of the list

Average Case Analysis

- We need a probability distribution for finding the key in position i.
- Assume every position is equally likely : This is called the Uniform distribution.
- The expected value of n for the uniform distribution can be expressed as:

$$f(n) = \sum_{i \in AllPossibilities} p_i * f_i(n)$$

Sequential Search: Average Case

Number of comparisons:

Assume x is in the list and probabilities x to be in positions

0, 1, …, (n-1) = 1/n

The expected number of comparisons is :

$$c_n = \frac{1}{n}1 + \frac{1}{n}2 + \frac{1}{n}3 + \ldots + \frac{1}{n}n = \frac{1}{n}(1+2+3+\ldots+n) = \frac{1}{n}\frac{n(n+1)}{2} = \frac{n+1}{2}$$

→ $c_n = O(n)$

Example: Sequential Search

- If n = 1000
  - Best case $c_{1000} = 1$
  - Worst case $c_{1000} = 1000$
  - Average case :

$$c_{1000} = \frac{n+1}{2} = \frac{1000+1}{2} = 500.5$$

To which case should we base our algorithm/Program design?

Remember: "Life teaches us to be always prepared for the worst!"

Simplifying the Complexity Analysis

- We know that it is very difficult to express the exact result of analysis in simple and comparable terms.
- We need to simplify analysis results further.
- Simplifications:
  - Ignore actual and abstract statement costs
  - *Order of growth* is a simple and comparable measure:
    - Highest-order term is what counts
      - Remember, we are doing asymptotic analysis
      - As the input size grows larger it is the highest order term that dominates the rest!

Reminder :Upper Bound Notation

- In general, a function f(n)
  is O(g(n)) if there exist positive constants c and n_0 such that
  $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- When we say f (n) = O(g(n))
  we really mean $f(n) \in O(g(n))$.

Example : $n^2 + 42n + 7 = O(n^2)$ means:

The function $f(n) = n^2 + 42n + 7$

is an element of the set $O(n^2)$

Theorem

- If f(n) is a polynomial time complexity function of order k and $a_k>0$ (the other coefficients may be positive or negative)

  f(n) = $a_k n^k + a_{k-1} n^{k-1} + \ldots + a_1 n + a_0$

  then

  $f(n)=O(n^k)$

Implication: Only the highest order term is important.

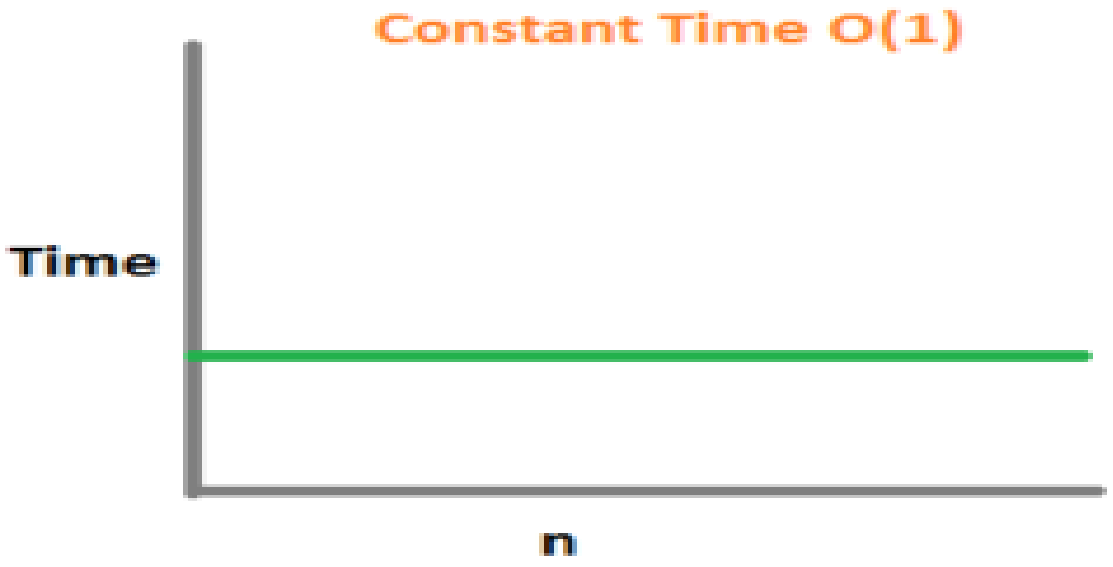
We can ignore the rest (lower order terms and coefficients).

Example : $f(n)= n^3 + 12\, n^2 - 8n + 45$

$=O(n^3)$

Constant Time

- Constant time implies that the number of operations the algorithm needs to perform to complete a given task is independent of the input size.



Examples :

- Accessing a single element of an array.
- Given a list, report the first element

Practice Problems

Question #1

The complexity of an algorithm A is $O(n^2)$.

For an input size of n=4, the algorithm runs for 10 ms

How long can you expect it to take to run on an input size of n=16?

Solution #1

The function is $f(n) = n^2$

So, 10 ms for the input of 4^2

How much time for 16^2

The proportion:

$$\begin{array}{cc} 4^2 & 10 \\ 16^2 & x \end{array}$$

$$(16^2 \times 10) / 4^2 = 160 \text{ ms}$$

Practice Problems

Question #2	Solution #2
The complexity of algorithm A is O(log n). For an input size of 16, the algorithm runs for 28 ms. How long can you expect it to take to run on an input size of 64?	The time needed for the input size 64 is proportional to log 64 So, the time needed for n=64 is $[\log_2 64 / \log_2 16]*28$ ms $= [6/4]*28$ $=42$ ms

Basic asymptotic efficiency classes

1	**constant**
log *n*	**logarithmic**
n	**linear**
n* log *n	**nlogn**
n^2	**quadratic**
n^3	**cubic**
2^n	exponential
$n!$	factorial